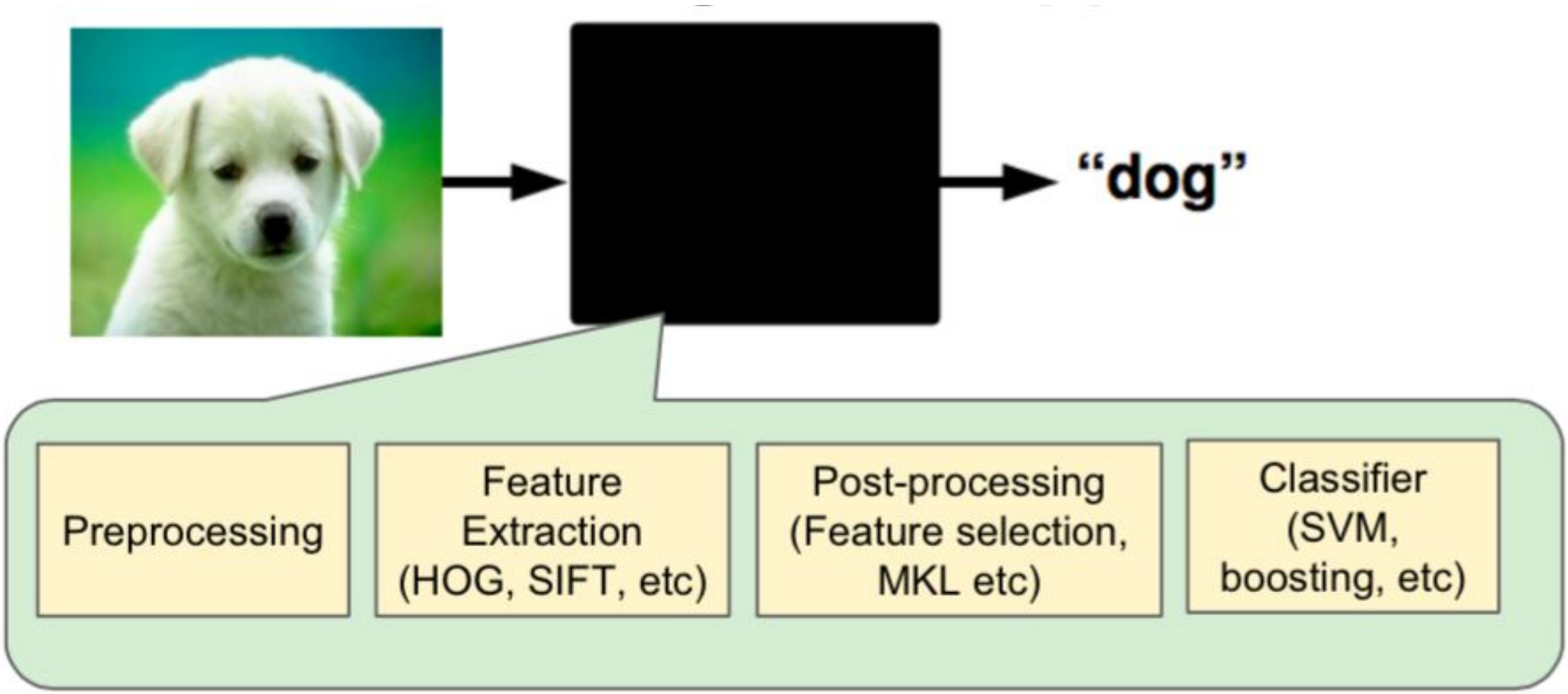
CNN - Supervised Classification

Soma Biswas
Department of Electrical Engineering,
Indian Institute of Science, Bangalore.

Slide Credit: Stanford Course: Convolutional Neural Network for Visual Recognition

Traditional Approach to Classification



What to do?

⇒ Learn an end-to-end recognition system.

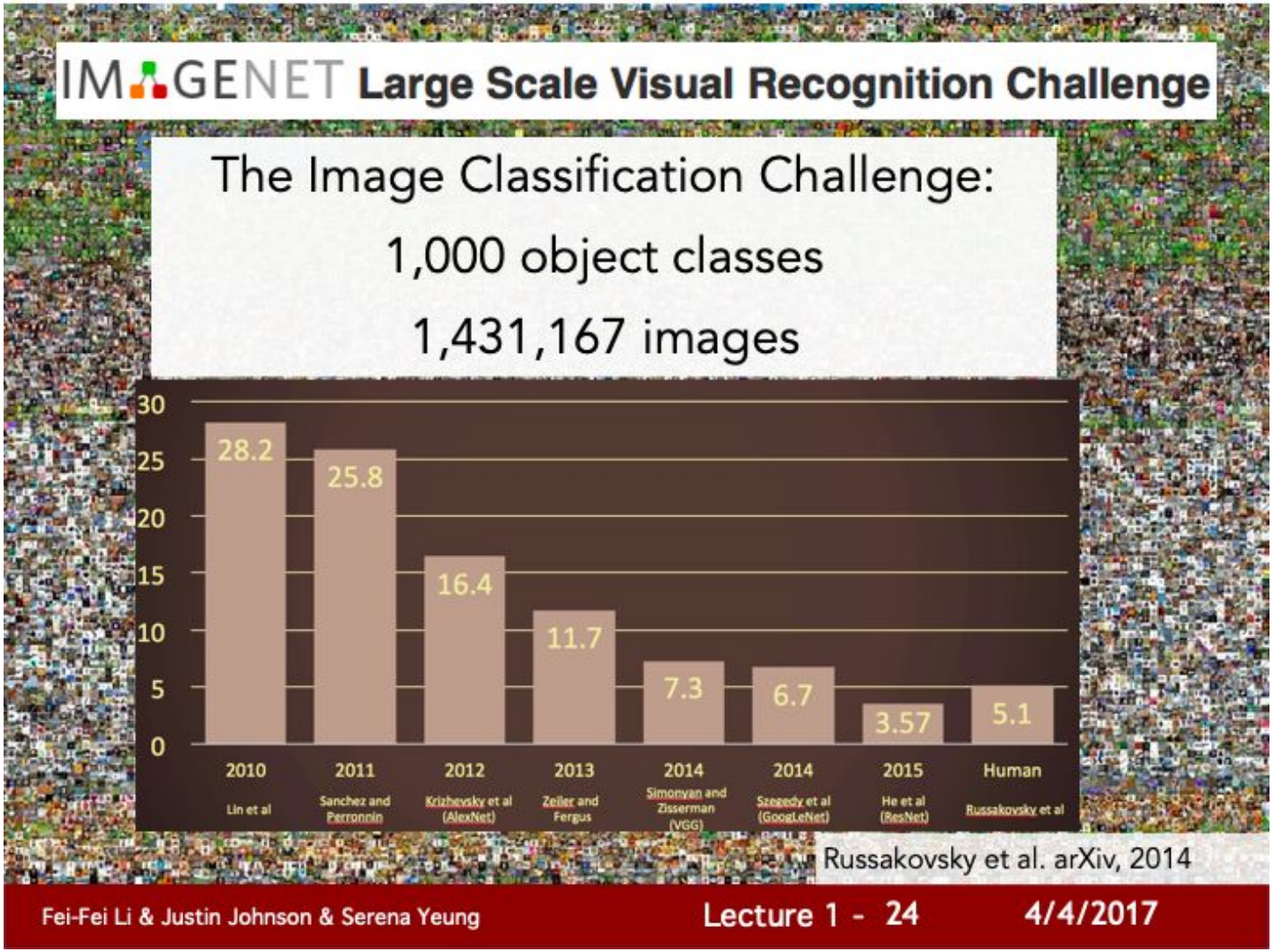
A non-linear map that takes raw pixels directly to labels.

Q: How can we build such a highly non-linear system?

A: By combining simple building blocks we can make more and more complex systems.

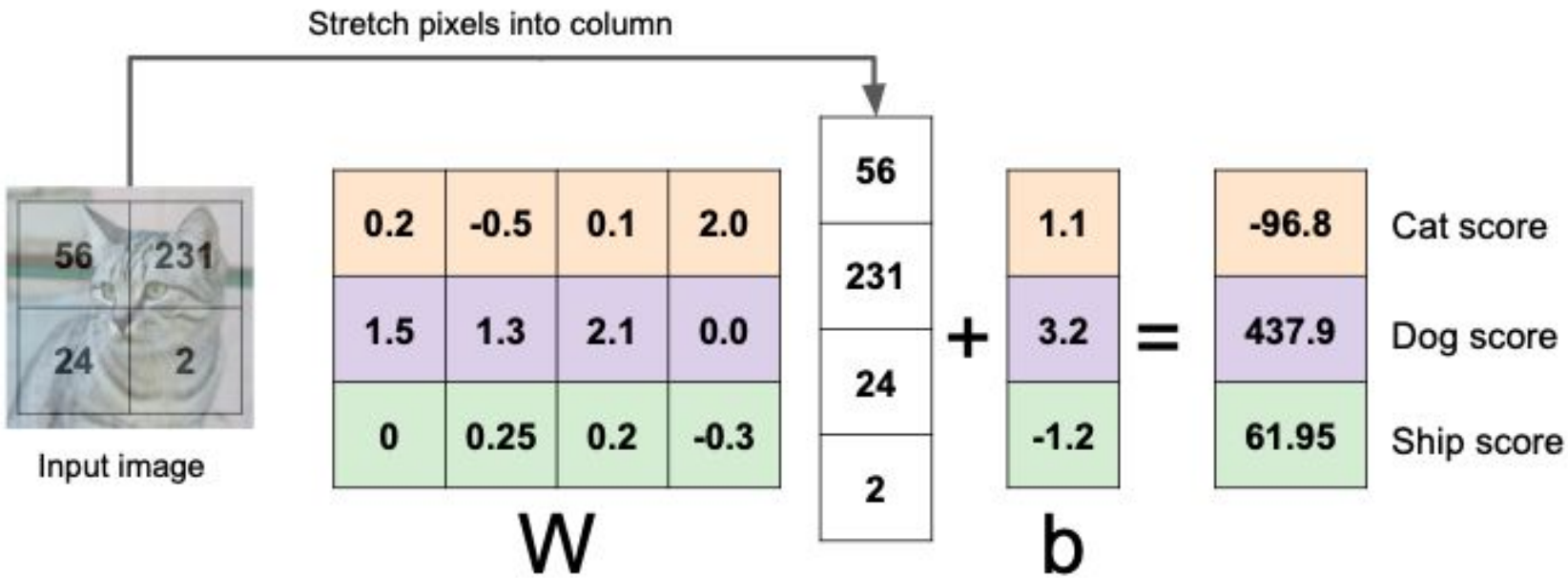
Slide: M. Ranzato

Starting of CNN

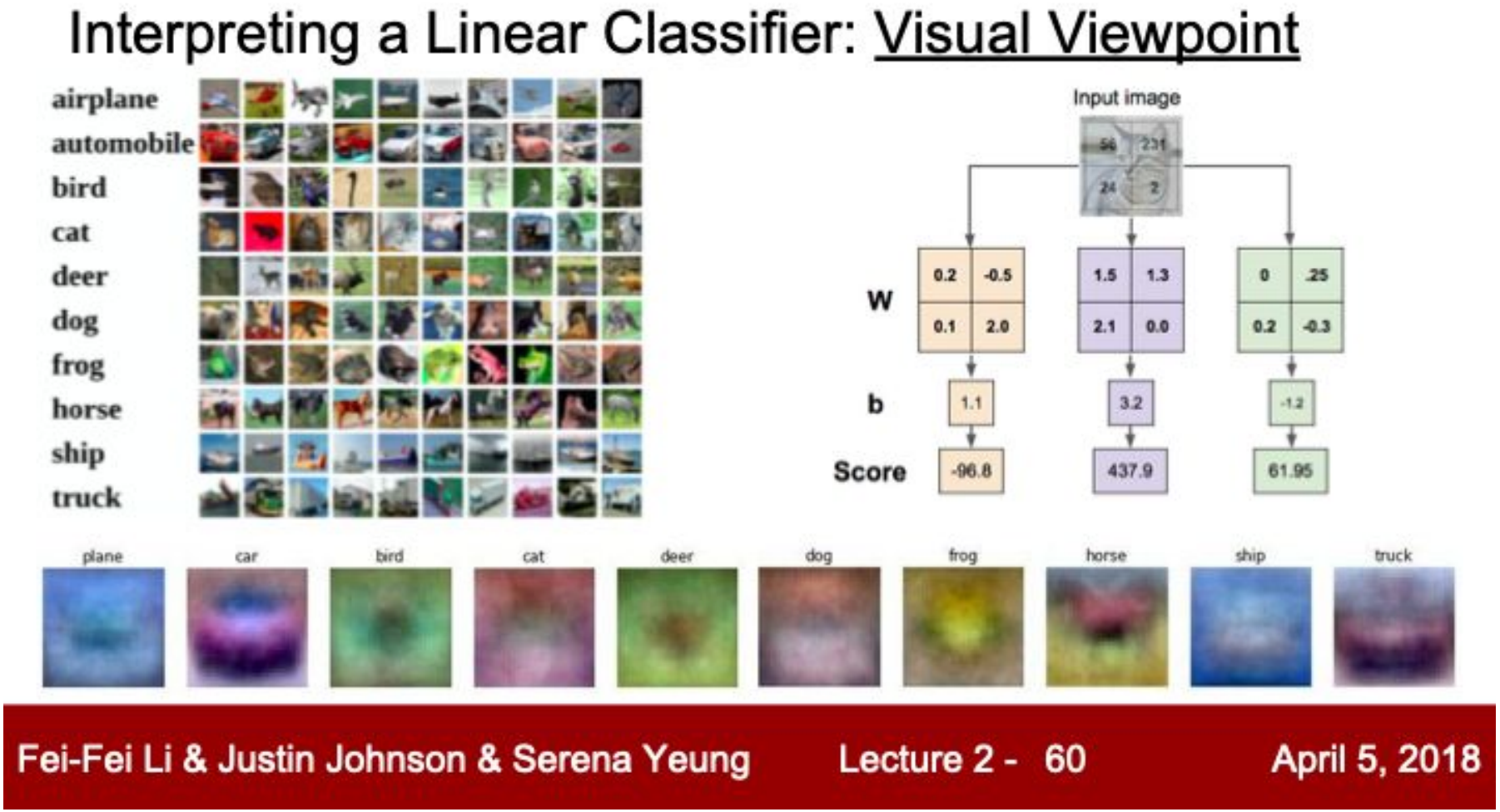


Linear Classifier

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



Templates


Interpreting a Linear Classifier: Visual Viewpoint
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
Input image
W
b
Score
0.2
-0.5
0.1
2.0
1.1
-96.8
1.5
1.3
2.1
0.0
3.2
437.9
0
.25
0.2
-0.3
-1.2
61.95
plane
car
bird
cat
deer
dog
frog
horse
ship
truck
Fei-Fei Li & Justin Johnson & Serena Yeung
Lecture 2 - 60
April 5, 2018

Loss Function and Optimization

- Define a **loss function** that quantifies our unhappiness with the scores across the training data.
- Come up with a way of efficiently finding the parameters that minimize the loss function. **(optimization)**

Loss Functions

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

A **loss function** tells how good our current classifier is

Given a dataset of examples

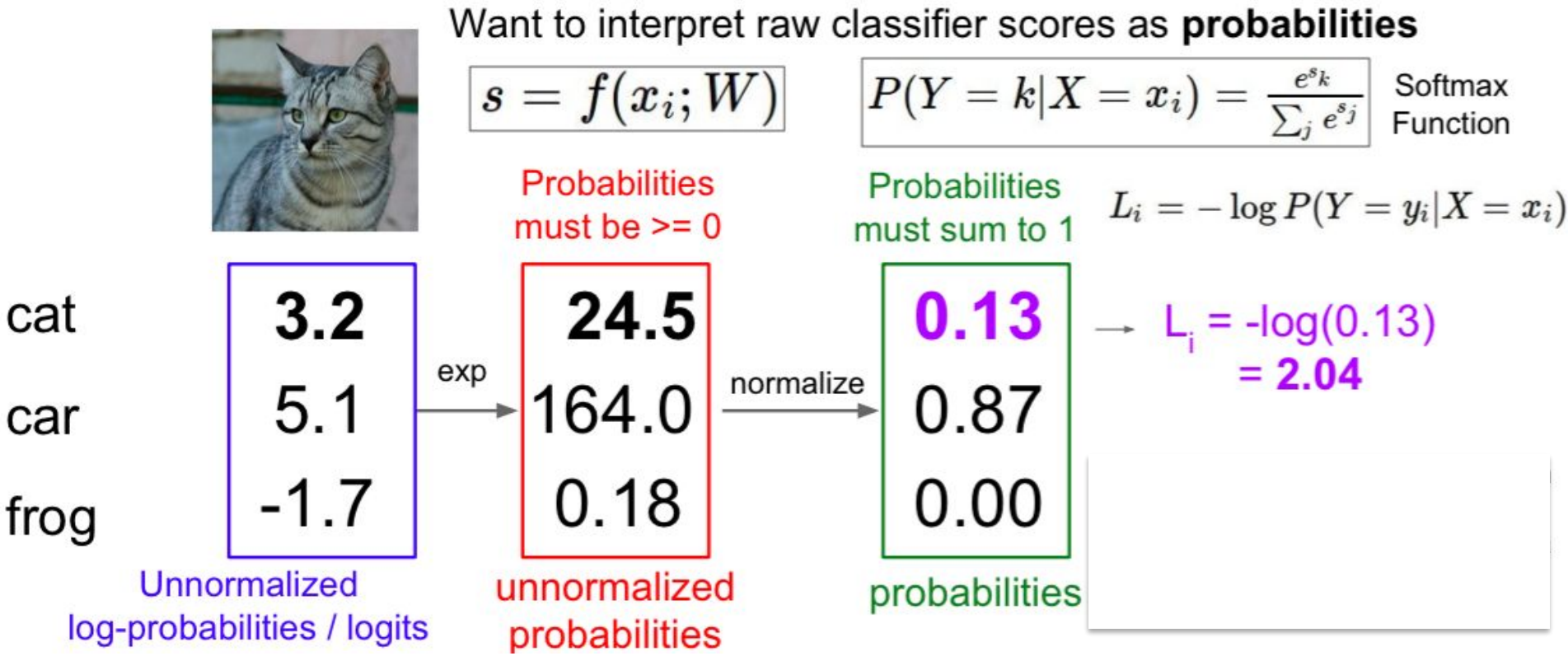
$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss over the dataset is a sum of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Softmax Classifier



- Want to maximize log likelihood or want to minimize negative log likelihood of the correct class

Cross-Entropy

- Cross entropy indicates the distance between what the model believes the output distribution should be, and what the original distribution really is.
- It is defined as, $H(y,p)=-\sum_i y_i \log(p_i)$
  - It is used when node activations can be understood as representing the probability that each hypothesis might be true, i.e. when the output is a probability distribution.
  - Thus it is used as a loss function in neural networks which have softmax activations in the output layer.

Regularization

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)} + \underbrace{\lambda R(W)}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Simple examples

<u>L2 regularization:</u> $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

More complex:

Dropout

Batch normalization

Stochastic depth, fractional pooling, etc

L2 Regularization

Regularization: Expressing Preferences

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = \boxed{[0.25, 0.25, 0.25, 0.25]}$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

L2 regularization likes to "spread out" the weights

How to find W? Optimization

- We have some dataset of (x,y)
- We have a **score function:** $s = f(x; W) \overset{\text{e.g.}}{=} Wx$
- We have a **loss function**:

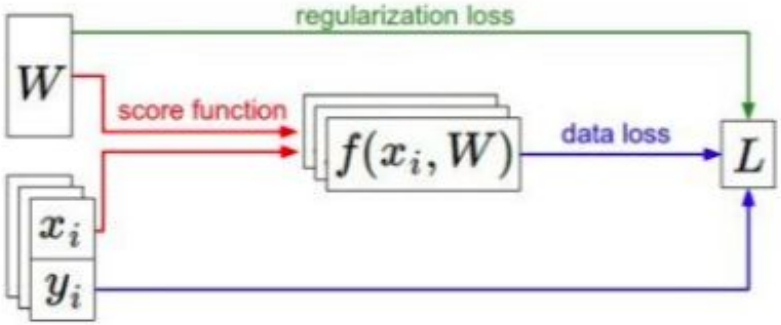
Softmax

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

SVM

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

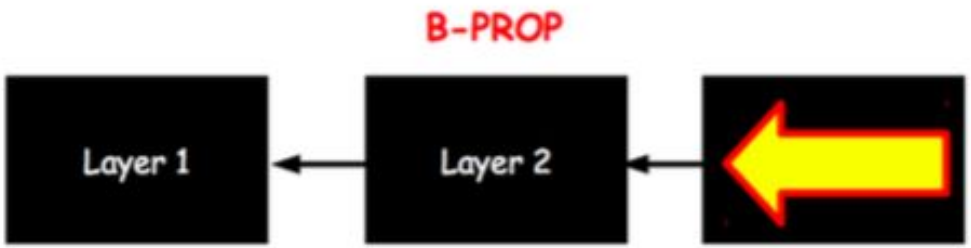
$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + R(W)$$ Full loss



Training

A) Compute loss on small mini-batch

A) Compute loss on small mini-batch
B) Compute gradient w.r.t. parameters



A) Compute loss on small mini-batch
B) Compute gradient w.r.t. parameters
C) Use gradient to update parameters $W \leftarrow W - \eta \frac{dL}{dW}$

Method 1: Follow the slope

want $\nabla_W L$

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

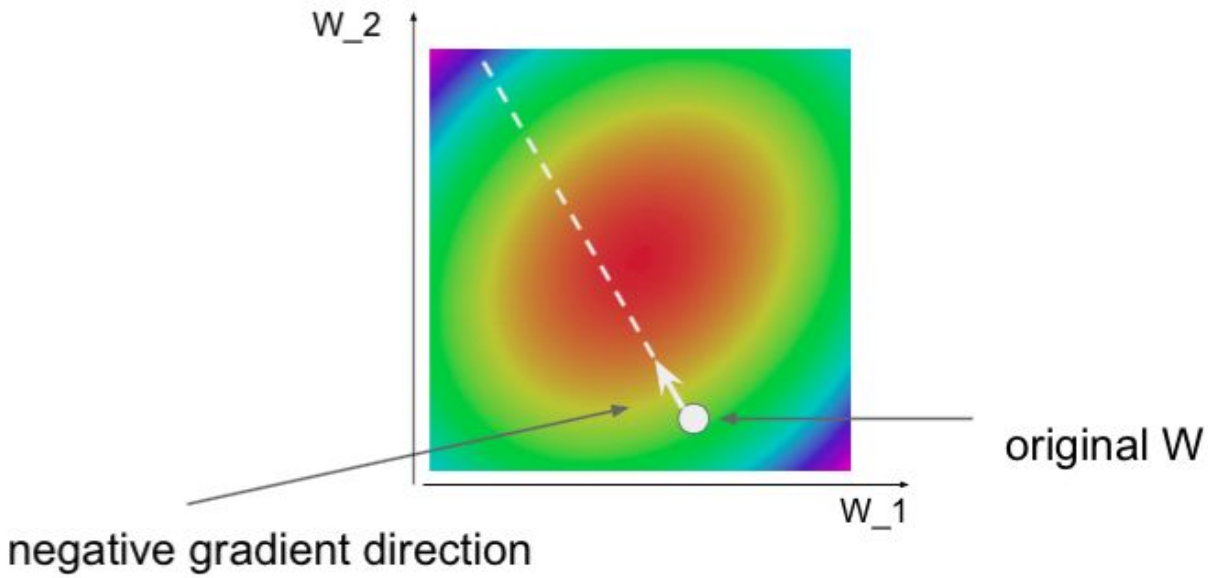
In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension

The slope in any direction is the **dot product** of the direction with the gradient
The direction of steepest descent is the **negative gradient**

Gradient Descent

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```



Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive when N is large!

Approximate sum using a **minibatch** of examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent

while True:
  data_batch = sample_training_data(data, 256) # sample 256 examples
  weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
  weights += - step_size * weights_grad # perform parameter update
```

Backpropagation – a simple example

Backpropagation: a simple example

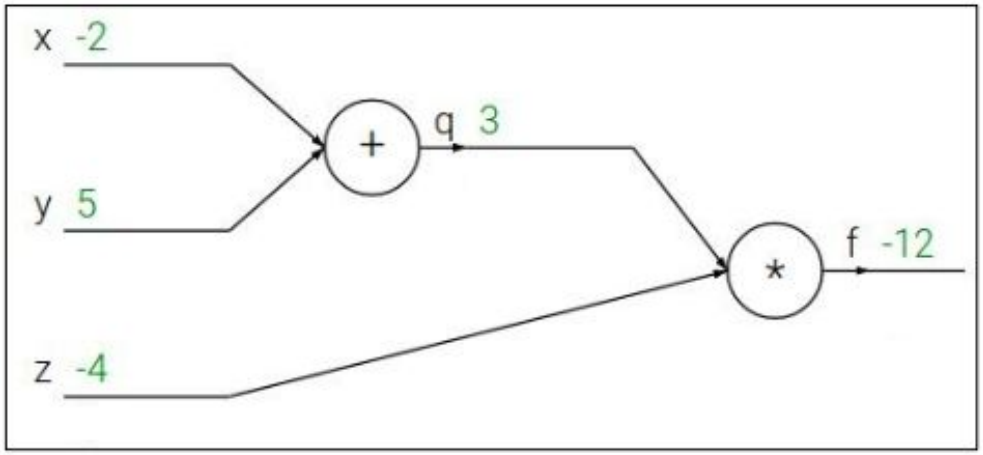
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation

Backpropagation: a simple example

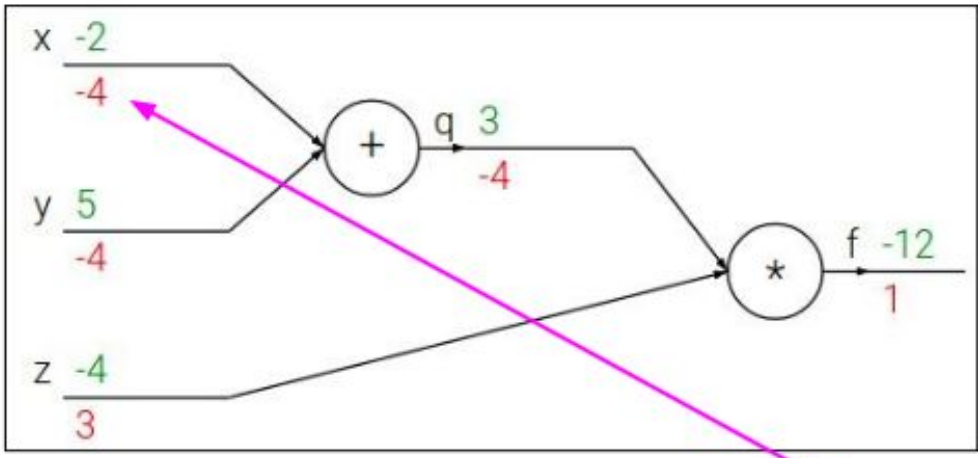
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

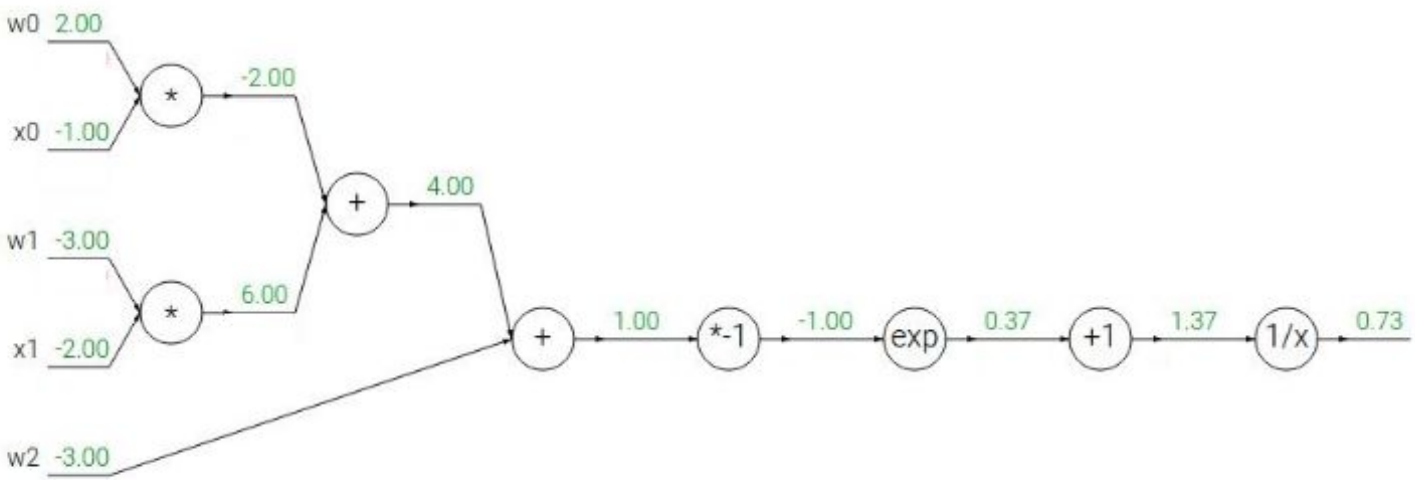
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Example

Another example: $$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

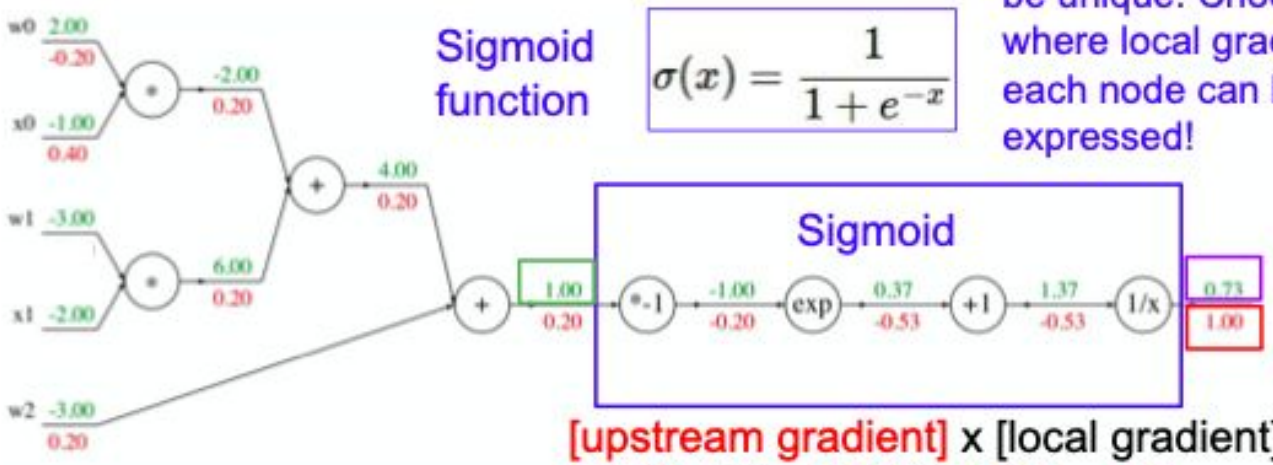
$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Contd.

Another example: $$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

Sigmoid function $$\sigma(x) = \frac{1}{1+e^{-x}}$$

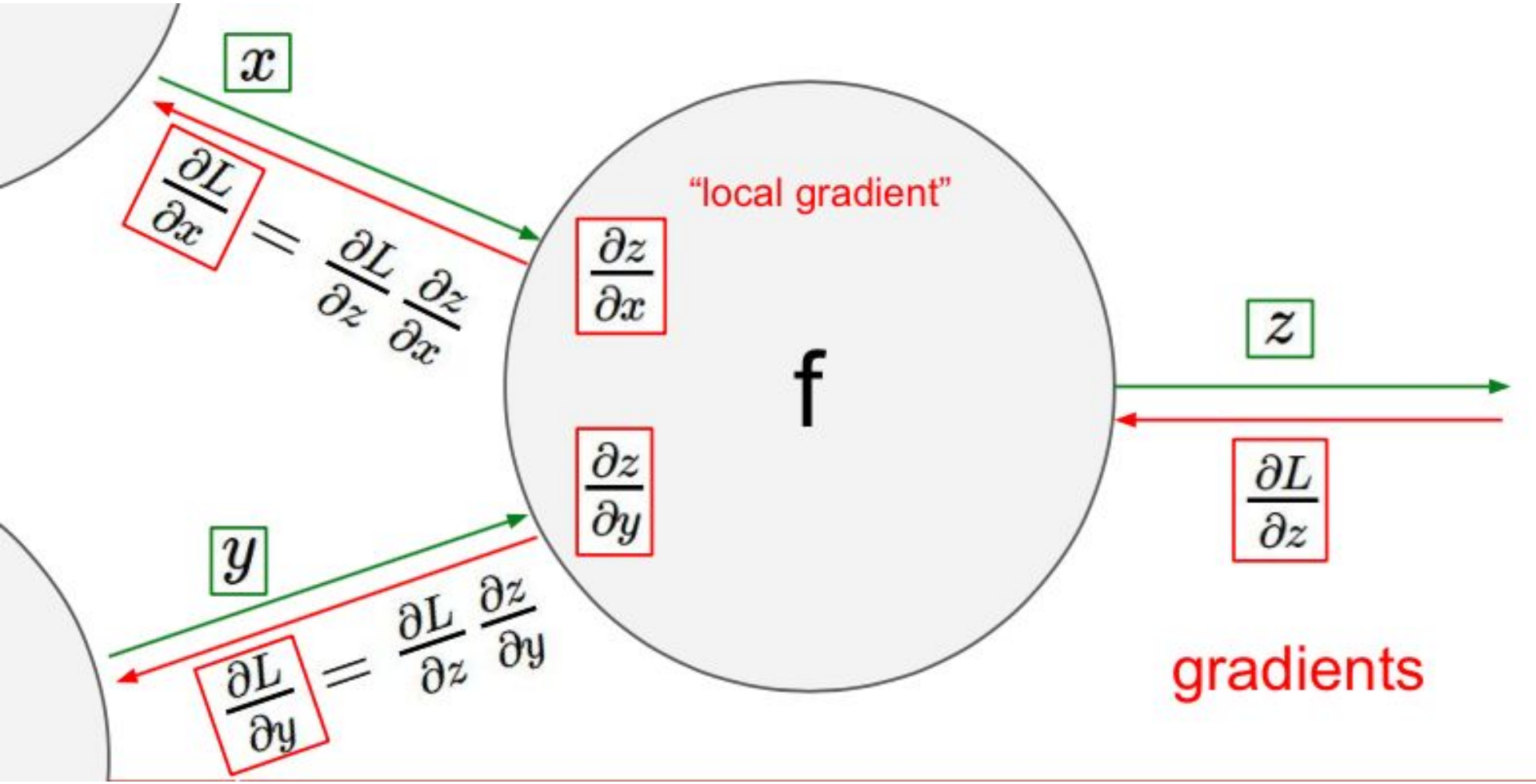


[upstream gradient] x [local gradient]

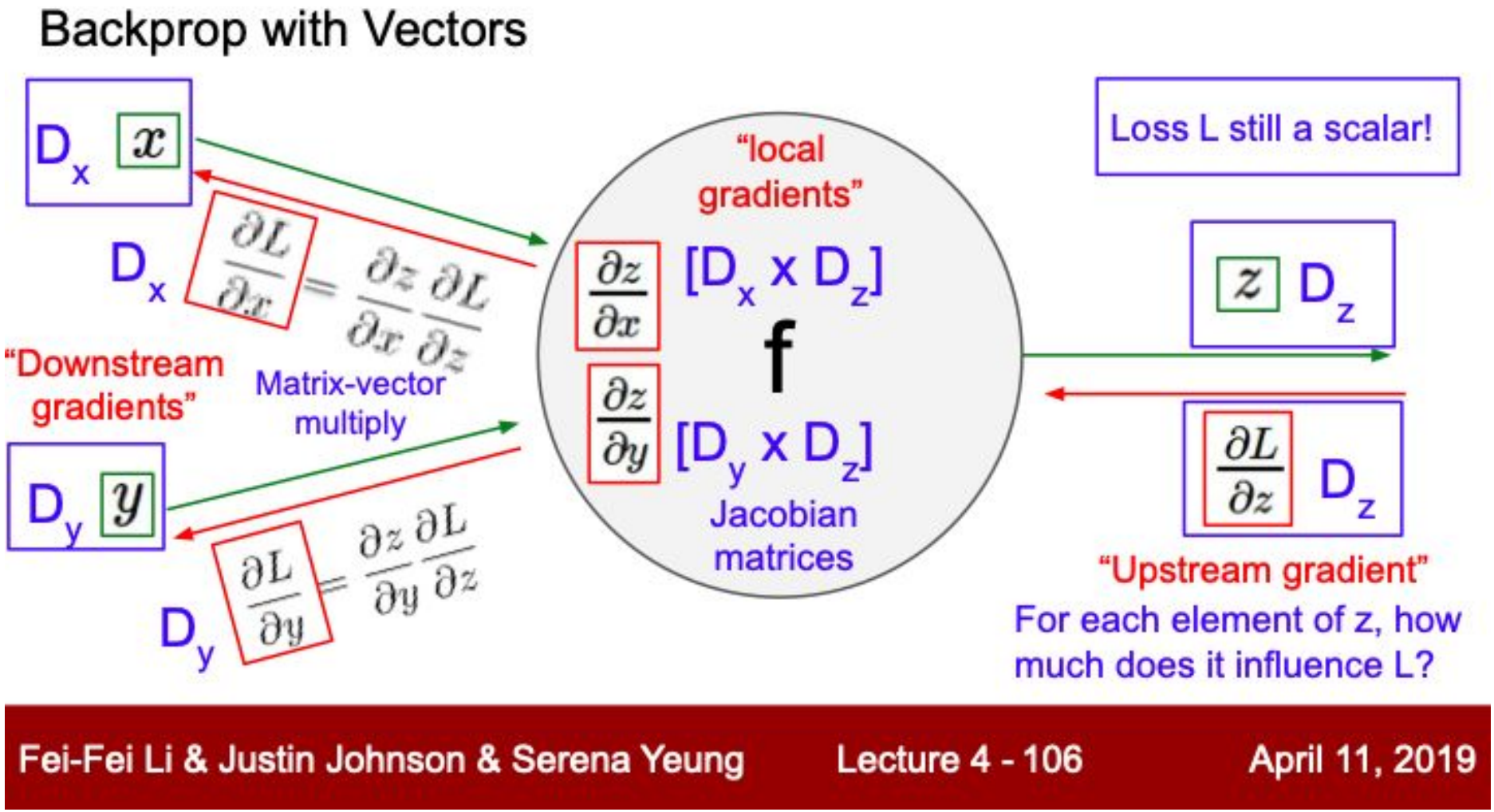
[1.00] x [(1 - 0.73) (0.73)] = 0.2

Sigmoid local gradient: $$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Gates


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"local gradient"
$\frac{\partial z}{\partial x}$
f
$\frac{\partial z}{\partial y}$
z
$\frac{\partial L}{\partial z}$
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
gradients

For Vectors


Backprop with Vectors
D_x x
D_x $\frac{\partial L}{\partial x} = \frac{\partial z}{\partial x}\frac{\partial L}{\partial z}$
"Downstream gradients"
Matrix-vector multiply
D_y y
D_y $\frac{\partial L}{\partial y} = \frac{\partial z}{\partial y}\frac{\partial L}{\partial z}$
"local gradients"
$\frac{\partial z}{\partial x}$ $[D_x \times D_z]$
f
$\frac{\partial z}{\partial y}$ $[D_y \times D_z]$
Jacobian matrices
Loss L still a scalar!
z D_z
$\frac{\partial L}{\partial z}$ D_z
"Upstream gradient"
For each element of z, how much does it influence L?
Fei-Fei Li & Justin Johnson & Serena Yeung
Lecture 4 - 106
April 11, 2019